

1. Approach Description

Our agent uses different strategies in the two phases of the game. First, in the placement phase, since both sides only make a very limited number of placement actions, and testing shows that the initial layout has a significant impact on the later game state, we do not directly use general search. Instead, we use a specialised line-opening heuristic with limited lookahead. The placement evaluator gives a small bonus to tested opening coordinates, but the final placement is still selected through heuristic scoring rather than by directly following a hard opening book. This helps the agent form a stable connected line or curve in the first few opening moves, while still keeping the opening phase fast through beam-style candidate filtering. Earlier versions used a more direct opening-book style based on tested opening lines, but local matches against the provided example agents and our benchmark versions showed that fixed openings were less reliable when the opponent chose unusual placements. Therefore, the final version changed to a score-based line-opening heuristic that still favours tested central coordinates, but also explicitly rewards connected line/curve formations and placements that reduce the opponent's future legal space. This scoring function considers support between our own pieces, whether they form a connected line or curve, whether the distance from the opponent is suitable, whether the placement reduces the opponent's future legal space, whether the cell is close to the edge, and whether the formation becomes too compact or isolated.

After entering the play phase, the agent uses negamax as the main search algorithm. Since Cascade is a game where the interests of the two players are completely opposed, it fully fits the definition of a zero-sum game. Therefore, we chose to use negamax, which is a simplified form of minimax for zero-sum games. To improve search efficiency, the agent also uses alpha-beta pruning to cut branches that do not affect the final choice. A normal minimax algorithm tries to search all legal actions under the current node, but in Cascade, each turn may include many Move, Eat, and Cascade actions, so the number of branches grows very quickly. This directly means that, without pruning, the agent would find it difficult to search to a deep enough level within the time limit.

In our algorithm implementation, each recursive search takes alpha and beta as inputs, and updates the current best score after searching the child nodes. If a node satisfies $\alpha \geq \beta$, it means the current branch can no longer affect the decision at the upper level, so the search stops immediately. To make pruning happen earlier, our agent also uses move ordering. It gives higher priority to promising actions such as Eat actions, positive-gain Cascade actions, the best action from the previous iteration, actions with high history scores, and cutting actions that have caused cutoffs before. In this way, strong actions are searched earlier, and the alpha or beta bounds can be tightened more quickly, which improves the practical effect of alpha-beta pruning.

We use iterative deepening to adapt to the time limit. The search starts from a shallow depth and gradually goes deeper, while keeping the best action from the previous fully completed depth. If the search gets close to the time limit, the agent returns the best result that has already been completed, instead of risking continuing the search. In addition, the agent first calculates a greedy fallback, ensuring that even if the search is interrupted, it can still return a legal and reasonable action.

Our agent's search also includes a transposition cache. During the search process, the agent encodes the board state as a state key and caches the search result at the corresponding depth. In this way, when the search tree reaches the same encoded board state through different action orders, the agent can reuse the previous result and reduce repeated computation.

Move ordering went through several rounds of optimization. Early versions only used simple sorting based on action type, such as searching Eat actions first, then Cascade actions, and then normal Move actions. This method was easy to implement, but it could only use very rough prior knowledge, and could not distinguish a high-gain Cascade from a self-damaging Cascade. Later versions began to simulate the result of each action before sorting, and scored each action based on the change in the game state before and after the action. For example, for an Eat action, the agent increases its priority based on the height of the stack being eaten. For a Cascade action, the agent calculates the token swing before and after the action, the opponent's token loss, our own token loss, and the change in the number of opponent stacks. In this way, Cascades that truly bring material advantage are searched first, while clearly losing Cascades do not receive high priority only because their type is Cascade. In the final version, move ordering further combines iterative deepening, the transposition cache, and history-based ordering. The best action found in the previous round of iterative deepening is tried first in the next round. If the transposition cache records the best action for a certain state, that action is also searched earlier. In addition, agent records cutting actions that have caused alpha-beta cutoffs before, and accumulates history scores for actions that performed well in the past.

In this way, the agent does not only sort actions based on the static result of the current move, but also uses experience accumulated during the search process to predict which actions are more likely to be effective. Although alpha-beta itself does not change the final negamax result, its efficiency depends heavily on the search order. The earlier strong actions are searched, the faster the alpha and beta bounds are tightened, and the easier it becomes to prune later weaker branches. Under the same time limit, this allows the agent to complete a deeper and more complete search.

We also added quiescence search to reduce the horizon effect. When the normal search reaches the depth limit, the agent does not stop immediately. Instead, it continues to search a small number of actions that can easily have a large impact on the game state, such as Eat actions and positive-gain Cascade actions. This helps avoid evaluating an unstable state too early when there is an obvious capture or cascade threat.

For the evaluation function, we use different evaluation weights for the first player and the second player, and we also add related tactical adjustments. This is because, through many rounds of testing, we found that they face different strategic problems in Cascade. RED is the first player, so it is usually easier for it to build an active structure and space advantage in the opening. However, it can easily waste its advantage through premature merging, self-damaging high-tower Cascades, or meaningless repetition when it is ahead. Therefore, RED's evaluation places more weight on structural safety, avoiding overmerge, avoiding high-value stacks being pushed off the board by Cascades, and continuing to stay active when ahead. BLUE is the second player, so in the opening it usually needs to respond to RED's active layout first. Its problem is therefore more about how to keep enough mobility

while defending, and how to turn small advantages into an actual win in the middle and late game. If BLUE only passively maintains the position, it can easily enter a draw or fail to expand its token advantage before the turn limit. Therefore, BLUE's evaluation places more weight on endgame conversion, avoiding repetition when ahead, increasing the value of Eat actions and positive-gain Cascades when the opponent has fewer tokens, and using repetition reasonably to seek a draw when behind. The evaluation features include token count, turn-limit token pressure, stack count, mobility, height power, tower shape, capture value, vulnerability, edge risk, push threat, push-off risk, center control, defense shape, cascade potential, concrete cascade action value, repetition, endgame conversion, and overmerge risk.

For empirical weight tuning, we use a lightweight feature-vector-based method inspired by TD-Leaf and temporal-difference value learning, but it is not a complete machine-learning strategy model. The agent represents the board state as a feature vector. We sample these feature vectors from local agent-vs-agent games, and construct target values based on the final win/loss result, or the token margin in the case of a draw. Then, the tuning scripts use gradient descent with L2 regularisation to fit separate evaluation weights for RED and BLUE. To avoid overfitting caused by the small-scale training data, the final weights do not completely replace the manually designed weights. Instead, they are conservatively blended with the original hand-designed weights, and the resulting versions are accepted only when local regression matches show stable improvement.

2. Performance Evaluation

We evaluated the agent mainly through local agent-vs-agent matches, rather than only testing a few isolated positions. During development, we kept several earlier versions as benchmark opponents, including random, greedy_v1, stable_backup, v2pro, v3, v5, v10, v11_eval, v14_tuned, v14_endgame, and v16_round2. For each important version, we tested both colour orders, because the agent's performance as RED and BLUE can be very different. RED moves first and can usually build initiative from the opening, while BLUE often needs to defend first and convert smaller advantages later. Therefore, only testing one side would give an incomplete evaluation.

For each match, we recorded the final result, win/draw/loss/error, average running time, final turn number, remaining token counts, and the reason for the game ending. These statistics helped us distinguish between different kinds of failures. For example, a draw with a large token advantage usually means the agent was not active enough in converting the advantage, while an early loss often means there was a tactical safety problem, such as allowing an immediate Eat or Cascade win for the opponent. Because the project performance is mainly based on winning games, we treated win rate as more important than simply having a material advantage or reaching many draws.

Across the later regression tests, our final deep3-style agent showed a clear improvement over the earlier v17-style agent. Table 1 summarises one local regression matrix for the final deep3-style agent against earlier benchmark versions. In this matrix, it achieved 115 wins out of 144 games, or about 79.9% overall. Its performance as RED was stronger, with 65 wins out of 72 games, or about 90.3%. Its BLUE result was weaker but still improved, with

50 wins out of 72 games, or about 69.4%. This matches our observations from development: the line-opening strategy made RED much better at turning the placement phase into a middle-game advantage, while BLUE still remained harder to tune because it needs to balance defence, mobility, and endgame conversion.

opponent	side	games	win	draw	loss	error	avg_sec
random	RED	10	10	0	0	0	40.7
random	BLUE	10	10	0	0	0	25.9
greedy_v1	RED	10	10	0	0	0	27.8
greedy_v1	BLUE	10	10	0	0	0	23.2
stable_backup	RED	10	10	0	0	0	90.3
stable_backup	BLUE	10	10	0	0	0	110.0
v2pro	RED	5	5	0	0	0	45.0
v2pro	BLUE	5	3	2	0	0	103.1
v3	RED	5	5	0	0	0	124.6
v3	BLUE	5	4	0	1	0	61.4
v5	RED	5	5	0	0	0	68.1
v5	BLUE	5	5	0	0	0	66.9
v10	RED	5	5	0	0	0	71.8
v10	BLUE	5	4	1	0	0	71.8
v11_eval	RED	5	5	0	0	0	74.8
v11_eval	BLUE	5	2	3	0	0	90.7
v14_tuned	RED	5	5	0	0	0	107.2
v14_tuned	BLUE	5	1	3	1	0	80.5
v14_endgame	RED	5	4	0	1	0	170.5
v14_endgame	BLUE	5	1	2	2	0	110.1
v16_round2	RED	7	1	5	1	0	98.6
v16_round2	BLUE	7	0	5	2	0	75.7

We selected the final version not only because it had the best overall win rate, but also because its behaviour was more stable and explainable. Some experimental versions, especially versions involving more aggressive Cascade logic or hybrid MCTS, could occasionally win difficult games, but they were less consistent and sometimes introduced new losses. By contrast, the final negamax-based version had fewer unexpected failures, converted more draw-heavy positions into wins, and still stayed within the time limit. Therefore, we chose it as the final agent because it gave the best balance between strength, stability, and simplicity.

3. Other Aspects

One important technical aspect of our work is that we did not rely only on the evaluation function to solve every problem. During testing, we found that some mistakes were too tactical to be fixed reliably by changing weights. For example, an action might look acceptable under the evaluation function, but after that move the opponent could immediately win by Eat or Cascade. To handle this, we added root-level safety guards for obvious terminal threats. These guards are only used to filter clearly losing actions at the root, while the normal negamax search still decides between the remaining legal actions. This made the agent safer without turning the whole program into a large set of hard-coded rules.

We also used different strategic adjustments for RED and BLUE. RED's main problem was how to use its first-player advantage to create pressure and then convert that pressure into a win. Therefore, the RED side was tuned to keep initiative, avoid self-damaging high-tower

Cascades, and avoid wasting an advantage through repetition. BLUE's problem was different: it often needed to survive RED's opening pressure, keep enough mobility, and avoid turning small advantages into draws. For this reason, some final changes were BLUE-only, such as stronger sensitivity to vulnerability and edge risk, an immediate terminal loss guard, and earlier endgame conversion when the opponent had few tokens left.

Another important optimisation was the action ordering system. At first, actions were ordered only by rough action type, such as trying Eat actions before Move actions. Later, we made the ordering more informed by simulating the result of each action before searching it. Eat actions are prioritised based on the value of the captured stack. Cascade actions are scored using token swing, opponent token loss, our own token loss, and the change in opponent stack count. The final version also combines this with iterative deepening, the transposition cache, previous best actions, and history-based scores. This does not change the theoretical result of alpha-beta search, but it makes pruning happen much earlier in practice.

We also considered MCTS and hybrid MCTS as an alternative enhancement, especially for complex middle-game positions where alpha-beta might not search deeply enough. However, in our tests, MCTS did not give a stable improvement. The rollout results were noisy unless the rollout policy was strong, and increasing the number or depth of simulations used too much time. It also made the agent's behaviour harder to explain and tune together with our existing evaluation function. Therefore, we decided not to use MCTS in the final version, and instead focused on improving the negamax framework that was already more stable.

Overall, one of our main design ideas was to keep the final agent strong but not unnecessarily complicated. We found that adding more features does not always make the player better. Some changes improved one matchup but damaged another. Therefore, in the final stage we tried to make small targeted changes, run regression tests after each change, and keep the parts that actually improved the win rate. This process led to a simpler and more stable final agent, based on negamax search, strong move ordering, practical safety guards, and colour-specific strategic tuning.

4. Supporting Work

For development and comparative testing, we implemented a series of helper tools to help us observe game results more quickly and correctly. First, we kept multiple agent versions as baselines and regression opponents. By letting these versions play against each other, we were able to judge whether each algorithm change brought a stable improvement.

We also wrote local log summary tools, such as `summarize_logs.py` and `summarize_v1_logs.py`. These scripts read the game logs generated by the referee, and extract the two agents, the final result, the ending turn number, the remaining token counts of both sides, and the reason for the game ending. In this way, we can quickly find whether a certain version often enters repetition draws, whether it keeps a token advantage before or around the turn limit, or whether one colour is clearly weaker than the other.

The tuning tools were written to support the design of the evaluation function. `collect_training_data.py` and `collect_training_data_dynamic.py` sample game states from

local agent-vs-agent games and export feature vectors, such as token advantage, mobility, capture value, vulnerability, cascade action value, repetition, overmerge risk, and endgame conversion. Then, `train_weights.py` and `train_weights_dynamic.py` use these samples to tune separate evaluation weights for RED and BLUE.

In addition, we used `diagnose_bias.py` and multiple sets of regression logs to analyse version bias. For example, when a certain version performed clearly differently as RED or BLUE, we checked the game logs and summaries to locate whether the problem came from the opening, repetition, endgame conversion, or RED's self-damaging high-tower Cascades. The RED safety logic, BLUE endgame conversion logic, and repetition-aware adjustment in the final agent were all gradually added through these diagnoses and regression tests.

References

AI Usage Statement: In this project, ChatGPT was used as an assistive tool to check and correct Python syntax errors, help locate code issues based on code snippets and game logs, suggest possible directions for code and algorithm optimisation, assist with empirical training/tuning workflows, help design scripts for running local self-play experiments, collecting match results, and summarising performance statistics, assist with improving the final code's cleanliness, expand and refine existing comments, and perform grammar checking and refinement for the report.